Then $\Pi_B(r_1)$ would be $\{(a), (a)\}$, whereas $\Pi_B(r_1) \times r_2$ would be

$$\{(a, 2), (a, 2), (a, 3), (a, 3), (a, 3), (a, 3)\}$$

We can now define how many copies of each tuple occur in the result of an SQL query. An SQL query of the form

select $A_1,\ A_2, \ldots, A_n$
from $r_1,\ r_2, \ldots, r_m$
where P

is equivalent to the relational-algebra expression

$$\Pi_{A_1,\ A_2,\ldots,A_n}(\sigma_P(r_1 \times r_2 \times \cdots \times r_m))$$

using the multiset versions of the relational operators σ, Π, and $\times$.

4.3 Set Operations

The SQL operations **union**, **intersect**, and **except** operate on relations and correspond to the relational-algebra operations $\cup$, $\cap$, and $-$. Like union, intersection, and set difference in relational algebra, the relations participating in the operations must be *compatible*; that is, they must have the same set of attributes.

Let us demonstrate how several of the example queries that we considered in Chapter 3 can be written in SQL. We shall now construct queries involving the **union**, **intersect**, and **except** operations of two sets: the set of all customers who have an account at the bank, which can be derived by

select *customer-name*
from *depositor*

and the set of customers who have a loan at the bank, which can be derived by

select *customer-name*
from *borrower*

We shall refer to the relations obtained as the result of the preceding queries as *d* and *b*, respectively.

4.3.1 The Union Operation

To find all customers having a loan, an account, or both at the bank, we write

(**select** *customer-name*
from *depositor*)
union
(**select** *customer-name*
from *borrower*)

The **union** operation automatically eliminates duplicates, unlike the **select** clause. Thus, in the preceding query, if a customer—say, Jones—has several accounts or loans (or both) at the bank, then Jones will appear only once in the result.

If we want to retain all duplicates, we must write **union all** in place of **union**:

(**select** *customer-name*
from *depositor*)
union all
(**select** *customer-name*
from *borrower*)

The number of duplicate tuples in the result is equal to the total number of duplicates that appear in both *d* and *b*. Thus, if Jones has three accounts and two loans at the bank, then there will be five tuples with the name Jones in the result.

4.3.2 The Intersect Operation

To find all customers who have both a loan and an account at the bank, we write

(**select distinct** *customer-name*
from *depositor*)
intersect
(**select distinct** *customer-name*
from *borrower*)

The **intersect** operation automatically eliminates duplicates. Thus, in the preceding query, if a customer—say, Jones—has several accounts and loans at the bank, then Jones will appear only once in the result.

If we want to retain all duplicates, we must write **intersect all** in place of **intersect**:

(**select** *customer-name*
from *depositor*)
intersect all
(**select** *customer-name*
from *borrower*)

The number of duplicate tuples that appear in the result is equal to the minimum number of duplicates in both *d* and *b*. Thus, if Jones has three accounts and two loans at the bank, then there will be two tuples with the name Jones in the result.

4.3.3 The Except Operation

To find all customers who have an account but no loan at the bank, we write